

1. Time in Microseconds shown.

Time in microseconds shown.

Time to create arrays is not counted. Only sorting.

Array Size	Merge	TMerge 2	TMerge 4	TMerge 16	TMerge 32
1	0	0	0	0	0
2	2	379	1	0	0
4	2	395	613	1	1
8	5	369	565	2	2
16	6	380	520	2496	6
32	17	384	402	3213	6308
64	38	468	621	3085	6069
128	76	489	837	5912	9703
256	165	1529	1074	3258	7546
512	401	604	704	3397	9769
1024	812	880	894	3489	7587
2048	1823	1267	1341	3170	5931
4096	4187	3155	2599	4443	8591
8192	6453	3816	2890	5347	10841
16384	12382	7543	4918	6375	11493
32768	27358	15418	10740	12803	19442
65536	53213	28252	21100	21106	36033
131072	107308	54076	37372	39124	57319
262144	211955	135289	90331	88737	123068
524288	453383	284870	184176	161948	223935
1048576	968520	523128	331456	299845	425519

justinn1508@DuyNguyenLanton:~/cs361/hw3\$

2. The algorithm improves the speed of sorting because the task is given even to thread and it will perform it parallel to each other.

3. The most difficult part of this project is the merging of the result of the thread. The task was quite challenging for me to implement the algorithm. I knew that I can make a better merge for threadsort but as I start late, I could only do the basics which is merging thread by thread.

4. The easiest part is the threaded merge function since we just need to divide the sort task to number of threads we have.